# Docker-Compose con Node, MySQL y Web

## 1. Crear el Servidor Web (Frontend)

Esta es la parte web para consumir la API de Node.js y mostrar los productos con estilos.
Vamos a crear un servidor en Express con EJS para que se vea más estructurado.

Crear el Proyecto Web

Dentro del mismo proyecto, crea una carpeta web y dentro de ella:

1. Crea un package.json para manejar dependencias:

   cd web

   npm init -y

2. Instala Express y EJS:

   npm install express ejs axios

3. Crea un server.js dentro de web/ con este código:

```
import express from 'express';
import axios from 'axios';

const app = express();

// Configurar EJS como motor de vistas
app.set('view engine', 'ejs');
app.set('views', './views');

// Ruta principal que obtiene los productos desde el backend
app.get('/', async (req, res) => {
    try {
        const response = await axios.get('http://backend:3000/productos');
```

```
        res.render('index', { productos: response.data });
    } catch (error) {
        console.error("Error obteniendo los productos:", error);
        res.status(500).send("Error obteniendo los productos.");
    }
});

// Puerto de escucha
app.listen(5000, () => {
    console.log("Servidor web corriendo en el puerto 5000");
});
```

El servidor web se crea para servir como interfaz de usuario, conectándose al backend para obtener los productos y mostrarlos de manera estructurada. Se usa Express con EJS porque Express facilita la creación de servidores en Node.js, y EJS es un motor de plantillas que permite insertar dinámicamente datos en el HTML.

---

### 2. Crear una Vista con EJS

En la carpeta web/, crea una carpeta llamada views/ y dentro crea el archivo index.ejs:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Danna - Productos</title>
    <style>
        body {
            font-family: Arial, sans-serif;
```

```
    background: linear-gradient(135deg, #4b0082, #4169e1);
    color: white;
    text-align: center;
    margin: 0;
    padding: 0;
    min-height: 100vh;
}
h1 {
    color: #00bfff;
    margin-top: 20px;
    font-size: 2.5rem;
}
.productos-container {
    display: flex;
    flex-wrap: wrap;
    justify-content: center;
    padding: 20px;
}
.producto {
    background: #1a1a2e;
    padding: 15px;
    margin: 10px;
    border-radius: 15px;
    border: 2px solid #4169e1;
    box-shadow: 0 0 12px rgba(0, 0, 139, 0.5);
    transition: transform 0.3s, box-shadow 0.3s;
    width: 220px;
```

```
            color: #add8e6;
        }
        .producto:hover {
            transform: scale(1.05);
            box-shadow: 0 0 20px #4169e1;
        }
        h2 {
            color: #00bfff;
            margin-bottom: 10px;
            font-size: 1.5rem;
        }
    </style>
</head>
<body>
    <h1>Lista de Productos</h1>
    <div class="productos-container">
        <% productos.forEach(producto => { %>
            <div class="producto">
                <h2><%= producto.nombre %></h2>
            </div>
        <% }); %>
    </div>
</body>
</html>
```

Se necesita una vista donde se muestren los productos obtenidos desde el backend. EJS se usa para generar dinámicamente la estructura HTML con los datos de productos que recibe del servidor web.

---

### 3. Configurar Docker-Compose para el Servidor Web

En docker-compose.yml, agrega un servicio **web** debajo de backend:

```
  web:
    build: ./web
    depends_on:
      - backend
    ports:
      - "5000:5000"
```

También, crea un Dockerfile dentro de web/:

```
FROM node:22

WORKDIR /app

COPY package.json .
RUN npm install

COPY . .

EXPOSE 5000

CMD ["node", "server.js"]
```

Docker-Compose se usa para gestionar y orquestar los contenedores de Docker. Al agregar el servicio "web" en el archivo docker-compose.yml, se asegura de que el servidor web pueda comunicarse con el backend y exponerse en el puerto 5000.

---

### 4. Levantar Todo con Docker

Ejecuta nuevamente:

```
docker-compose up --build -d
```

Y prueba en el navegador:

http://localhost:5000

Con el comando docker-compose up --build -d, se inician todos los servicios definidos en el archivo docker-compose.yml. Esto configura tanto el servidor web como el backend, y lo ejecuta en segundo plano.